(domain) using the `host` attribute. In this case, the anchor tag helper will generate an absolute URL using the value(s) specified.

The final attribute in the list is the `area` attribute, which is used to specify the *area* the target page is in. The name of the `area` is included as the first segment of the generated URL.

Areas in Razor Pages

The `areas` feature in Razor Pages is a strange one in that the most important thing to know about it is that you should generally avoid using it. I wouldn't bother mentioning `areas` in this book at all, except that the default template includes anchor tag helpers that feature the `area` attribute (albeit, they have an empty string assigned to them) and areas are used within the Identity framework, which you will work within chapter 9.

Areas originate from the MVC framework. One of the problems with the MVC framework is the convention placing your controller classes in a folder named Controllers and the view files in a folder named Views—a convention that is enforced by the automated code scaffolding system. In a large application, you can end up with huge numbers of controller class files in this folder. The `areas` feature was devised to help break up large applications into separate subapplications. Each area in an MVC project has its own Controllers, Views, and Models folders. The contents of an area are reached by placing the name of the area as the first segment in the URL, so the area also implements a hierarchy for the purposes of routing. You can achieve exactly the same hierarchical routing effect in Razor Pages by adding a new subfolder to the Pages folder, which is generally why `areas` are not recommended within Razor Pages. They add significant complexity to a more easily resolved problem.

The primary reason for enabling `areas` in Razor Pages is that they facilitate Razor class libraries—another feature that will not be covered in detail in this book. If you are interested in learning more about Razor class libraries, you can refer to the official documentation (http://mng.bz/o2zd). The Identity framework includes a sample UI, which is implemented as a Razor class library. So while you won't add any areas of your own, you may well find yourself having to work with `areas` if you want to customize the Identity UI.